PARCIAL LENGUAJES DE PROGRAMACIÓN Y TRANSCRIPCIÓN #2

JULIO CESAR FLOREZ GUARNIZO

JOAQUIN FERNANDO SANCHEZ CIFUENTES

UNIVERSIDAD SERGIO ARBOLEDA

Ejercicio 1 y 2

Se diseñó e implementó un lenguaje de consulta tipo NoSQL que permite realizar operaciones CRUD (Create, Read, Update, Delete).
La gramática fue definida formalmente y posteriormente implementada utilizando ANTLR, generando un analizador sintáctico capaz de validar la estructura de las sentencias.

Definición del lenguaje

El lenguaje permite:
- Crear documentos
- Consultar información
- Actualizar registros
- Eliminar registros

Ejemplos de uso

```
CREATE users {name: "Juan", age: 20}
READ users WHERE age = 20
UPDATE users SET age = 21 WHERE name = "Juan"
DELETE users WHERE age = 21
```

Gramática formal

S → stmt

stmt → create | read | update | delete

create → CREATE ID object
read → READ ID where_clause?
update → UPDATE ID SET assignments where_clause?
delete → DELETE ID where_clause?

where_clause → WHERE condition
condition → ID '=' value

*assignments → assignment (',' assignment)**
assignment → ID '=' value

object → '{' pair (',' pair) '}'*
pair → ID ':' value

value → STRING | NUMBER

Implementación en ANTLR (Punto 2)

Herramientas utilizadas

- ANTLR4
- Java
- macOS (Apple Silicon)
- Terminal

Gramática en ANTLR

```
grammar CRUD;

prog: stmt+ ;

stmt
    : create
    | read
    | update
    | delete
    ;

create
    : 'CREATE' ID object
    ;

read
    : 'READ' ID where_clause?
    ;

update
    : 'UPDATE' ID 'SET' assignments where_clause?
    ;

delete
    : 'DELETE' ID where_clause?
    ;

where_clause
    : 'WHERE' condition
    ;

condition
    : ID '=' value
    ;

assignments
    : assignment (',' assignment)*
    ;

assignment
    : ID '=' value
    ;

object
    : '{' pair (',' pair)* '}'
    ;

pair
    : ID ':' value
    ;

value
    : STRING
    | NUMBER
    ;

ID: [a-zA-Z_][a-zA-Z0-9_]* ;
NUMBER: [0-9]+ ;
STRING: '"' .*? '"' ;

WS: [ \t\r\n]+ -> skip ;
```

Generación del parser

antlr4 CRUD.g4
javac *.java

Pruebas del lenguaje

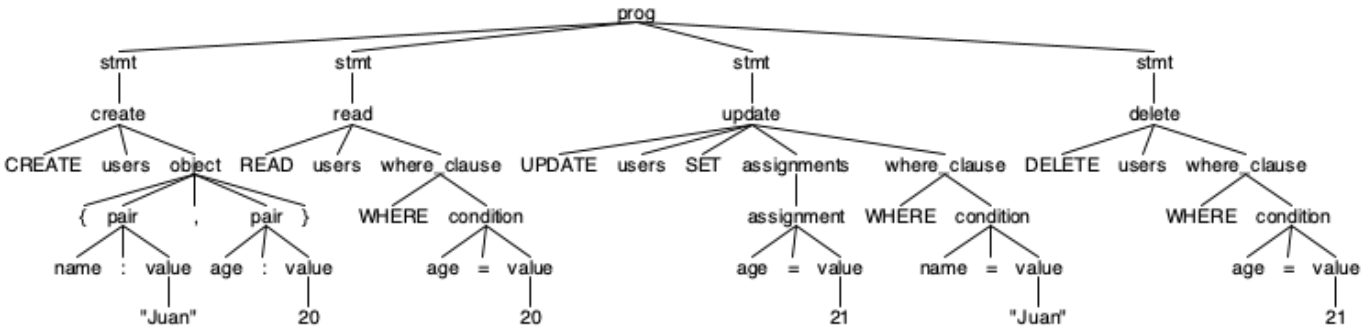
Entrada:

CREATE users {name: "Juan", age: 20}
READ users WHERE age = 20
UPDATE users SET age = 21 WHERE name = "Juan"
DELETE users WHERE age = 21

Salida

(prog (stmt (create CREATE users (object { (pair name : (value "Juan")) , (pair age : (value 20)) }))) (stmt (read READ users (where_clause WHERE (condition age = (value 20))))) (stmt (update UPDATE users SET (assignments (assignment age = (value 21))) (where_clause WHERE (condition name = (value "Juan"))))) (stmt (delete DELETE users (where_clause WHERE (condition age = (value 21))))))

El árbol de derivación muestra que cada sentencia es reconocida correctamente por el analizador sintáctico.



Punto 3

Se implementó un algoritmo completo en Python, sin uso de librerías externas, que permite calcular los conjuntos FIRST y FOLLOW a partir de la gramática original, incluyendo producciones con ε.

Posteriormente, se construyó la tabla predictiva LL(1), verificando la ausencia de conflictos en sus entradas.

```
=== TABLA LL(1) ===
M[S, a] = ['A', 'a', 'A', 'b']
M[S, b] = ['B', 'b', 'B', 'a']
M[A, b] = ['ε']
M[A, a] = ['ε']
M[B, b] = ['ε']
M[B, a] = ['ε']
```

Los resultados obtenidos confirman que la gramática es LL(1), lo cual coincide con el análisis teórico realizado previamente.

```
=== GRAMÁTICA ===
S → [['A', 'a', 'A', 'b'], ['B', 'b', 'B', 'a']]
A → [['ε']]
B → [['ε']]

=== FIRST ===
S : {'b', 'a'}
A : {'ε'}
B : {'ε'}

=== FOLLOW ===
S : {'$'}
A : {'b', 'a'}
B : {'b', 'a'}

=== TABLA LL(1) ===
M[S, a] = ['A', 'a', 'A', 'b']
M[S, b] = ['B', 'b', 'B', 'a']
M[A, b] = ['ε']
M[A, a] = ['ε']
M[B, b] = ['ε']
M[B, a] = ['ε']

=== RESULTADO FINAL ===
✔ La gramática ES LL(1)
```

Punto 4

Se realizó una comparación entre un parser predictivo (LL) y el algoritmo CYK, con el objetivo de analizar su rendimiento en términos de tiempo de ejecución.

Ambos parsers fueron implementados en Python y evaluados utilizando cadenas del lenguaje $a^n b^n$, incrementando progresivamente el tamaño de entrada.

Para cada valor de n, se generó una cadena de la forma $a^n b^n$.
Se midió el tiempo de ejecución de cada parser utilizando la función de medición de tiempo del sistema.

Los valores evaluados fueron n = 5, 10, 20, 30 y 40.

Resultados

n	Tiempo LL (s)	Tiempo CYK (s)
5	0.000006	0.000362
10	0.000008	0.001646
20	0.000012	0.013171
30	0.000019	0.031046
40	0.000017	0.051090

Los resultados muestran que el parser LL mantiene tiempos de ejecución prácticamente constantes, incluso al aumentar el tamaño de la entrada.

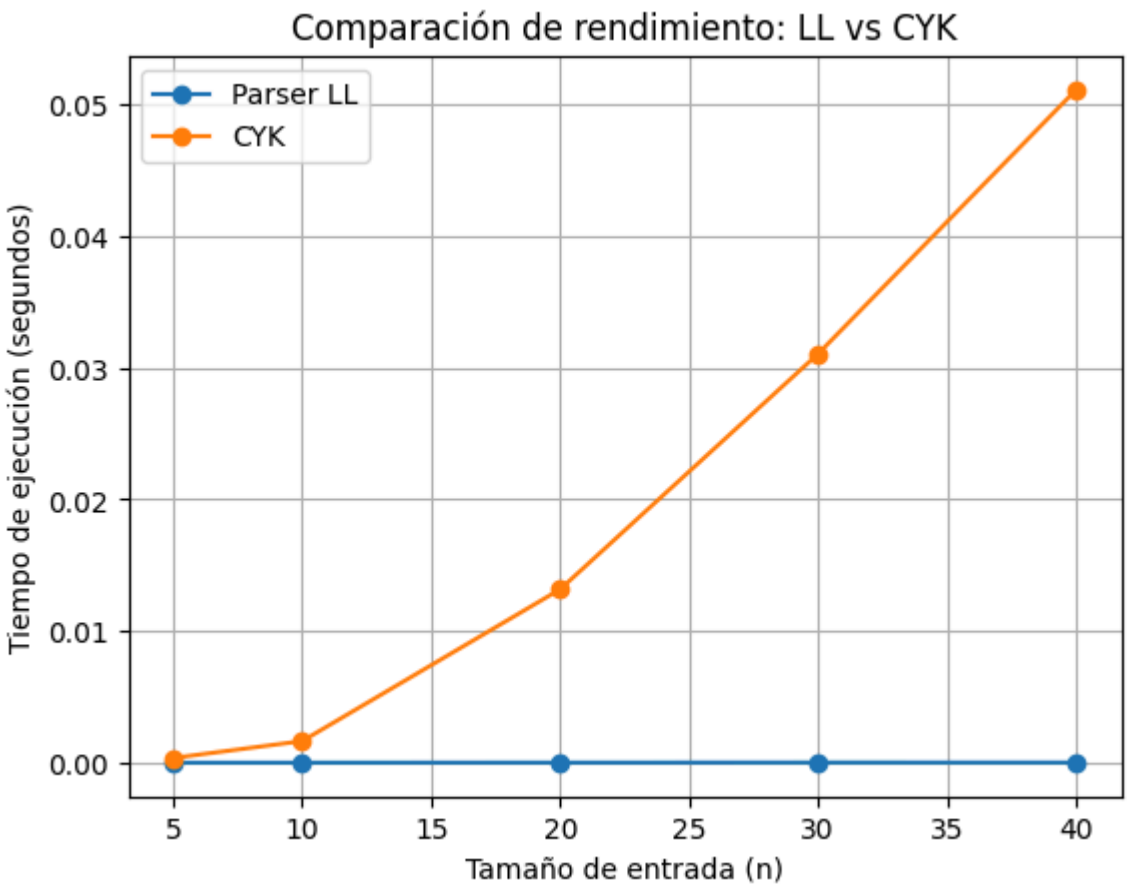


Figura X. Comparación del tiempo de ejecución entre el parser LL y el algoritmo CYK.

La gráfica muestra claramente que el tiempo del parser LL crece de forma casi constante, mientras que el algoritmo CYK presenta un crecimiento acelerado.
Esto visualmente confirma la diferencia entre complejidad lineal y cúbica.

En contraste, el algoritmo CYK presenta un crecimiento significativo en el tiempo de ejecución. Por ejemplo, al pasar de n=10 a n=40, el tiempo aumenta más de 30 veces.

Además, para n=40, CYK es aproximadamente 3000 veces más lento que el parser LL, lo que evidencia una diferencia considerable en eficiencia.

El comportamiento observado coincide con la teoría:

- El parser LL tiene complejidad O(n), lo que explica su crecimiento lineal.
- El algoritmo CYK tiene complejidad $O(n^3)$, lo que justifica el incremento acelerado en los tiempos.

Esto demuestra que, aunque CYK es más general, su costo computacional es significativamente mayor.

Se concluye que el parser predictivo (LL) es considerablemente más eficiente para lenguajes que cumplen con las restricciones necesarias.

Por otro lado, el algoritmo CYK, aunque más general y capaz de analizar cualquier gramática en forma normal de Chomsky, presenta un rendimiento inferior debido a su mayor complejidad computacional.

Punto 5

En este punto se desarrolló una calculadora de expresiones booleanas utilizando las herramientas YACC (Bison) y Flex.
El objetivo fue implementar un analizador sintáctico capaz de evaluar expresiones lógicas que incluyan operadores como AND, OR y NOT, así como el uso de paréntesis.

El sistema permite procesar múltiples expresiones y generar su resultado de forma inmediata, integrando análisis léxico, sintáctico y evaluación semántica.

Gramática definida

exp → exp AND exp
exp → exp OR exp
exp → NOT exp
exp → (exp)
exp → TRUE
exp → FALSE

Para resolver ambigüedades, se definió precedencia:

NOT > AND > OR

Ejecución

```
juli@Felipes-MacBook-Pro-2 ~ % ./calc
Ingrese expresiones booleanas:
true and false
Resultado: 0
true and false
Resultado: 0
true or false
Resultado: 1
not true
Resultado: 0
true and (false or true)
Resultado: 1
true and (false or true)
Resultado: 1
```

El sistema detecta errores sintácticos mediante la función yyerror, permitiendo identificar expresiones mal formadas.

Ejemplo:

Entrada:
true and

Salida:
Error: syntax error

El analizador sintáctico generado por YACC es de tipo LALR(1), lo que permite procesar las expresiones en tiempo lineal O(n).

Durante la ejecución, el parser utiliza una pila para manejar los estados y realizar operaciones de desplazamiento (shift) y reducción (reduce).
Las acciones semánticas asociadas a cada producción permiten evaluar las expresiones booleanas en tiempo constante por operación.
En conjunto, el sistema presenta un desempeño eficiente, adecuado para la evaluación de expresiones en tiempo real.